# Syntax

The syntax is the set of rules which defines the arrangements of symbols. Every language specification has its syntax. Syntax applies to **programming languages** in which the document represents the **source code** and also applies to **markup languages ,** in which the document describes the **data.**

The program within the JavaScript consists of:

**Literals:** A literal can be defined as a notation for representing the fixed value within the source code. Generally, literals are used for initializing the variables. In the following example, you can see the use of literals in which **1** represents the **integer literal,** and the string **"hello"** is a string**literal**.

```
1. int x = 1;
2. string str = "hello";
```

The ***syntax*** of computer programming language is utilized to represent the ***structure of programs*** without viewing their meaning. It mainly focuses on the structure and arrangement of a program with the help of its appearance. It consists of a set of rules and regulations that validates the sequence of symbols and statements that are utilized in a program. Both human languages and programming languages rely on syntax, and the pragmatic and computation model represents these syntactic elements of a computer programming language.

## Techniques of Syntax

There are several formal and informal techniques that may help to understand the syntax of computer programming language. Some of those techniques are as follows:

**1. Lexical Syntax**

It is utilized to define basic symbols' rules, including identifiers, punctuators, literals, and operators.

**2. Concrete Syntax**

It describes the real representation of programs utilizing lexical symbols such as their alphabet.

# Semantics

***Semantics*** is a ***linguistic concept*** that differs from syntax. In a computer programming language, the term semantics is utilized to determine the link between the model of computation and the syntax. The concept behind semantics is that linguistic representations or symbols enable logical outcomes because a collection of words and phrases communicates ideas to machines and humans. The syntax-directed semantics approach is utilized to map syntactical concepts to the computational model via a function.

## Techniques of Semantics

There are several techniques that may help to understand the semantics of computer programming language. Some of those techniques are as follows:

**1. Algebraic semantics**

It analyzes the program by specifying algebra.

**2. Operational Semantics**

After comparing the languages to the abstract machine, it evaluates the program as a series of state transitions.

**3. Translational Semantics**

It mainly concentrates on the methods that are utilized for translating a program into another computer language.

**4. Axiomatic Semantics**

It specifies the meaning of a program by developing statements about an establishment that detain at every stage of the program's execution.

**5. Denotational Semantics**

It represents the meaning of the program by a set of functions that work on the program state.

## Types of Semantics

There are several types of semantics. Some of those are as follows:

**1. Formal Semantics**

Words and meanings are analyzed philosophically or mathematically in formal semantics. It constructs models to help define the truth behind words instead of considering only real-world instances.

**2. Lexical Semantics**

It is the most well-known sort of semantics. It searches for the meaning of single words by taking into consideration the context and text surrounding them.

**3. Conceptual Semantics**

The dictionary definition of the word is analyzed before any context is applied in conceptual semantics. After examination of the definition, the context is explored by searching for linking terms, how meaning is assigned, and how meaning may change over time. It can be referred to as a sign that the word conveys context.

| Features | Syntax | Semantics |
|---|---|---|
| **Definition** | It is a program's structure written in a computer programming language. | It defines the meaning of a programmir language's related line of code. |
| **Basic** | It allowed phrases of a language. | It refers to the interpretation of th phrases. |
| **Relation** | The meaning of syntax interpretation should be unique. | A syntactic form is linked with semantic component. |
| **Errors** | Its errors are handled at the compile time. | Semantic errors are confronted runtime. |
| **Finding Errors** | Syntax errors are easy to find. | Semantic errors are complex to find. |
| **Sensitivity** | It is sensitive in most computer programming languages. | It is case-insensitive in some cases. |
| **In linguistics** | It is the arrangement or order of words that is determined by the writer's style as well as grammar rules. | Logical semantics and lexical semanti are two branches of semantics. |

## Pragmatics in Programming Languages

**Pragmatism, as a term, refers to treating a task in a manner that focuses on the practical aspect of the approach, to maximize efficiency. In the field of programming, this term refers to the "best practices" of programming. These often refer to writing clean code and managing the code in a manner as efficient as possible, to make it easily understandable, by the person that writes the code, and by the people that will read the code in the future.**

## Syntax

Is the set of rules that defines the combinations of symbols that are considered to be valid in a given language. It is about the structure or grammar of the language. Some JavaScript syntax rules are:

- if you put `var` before a word that word becomes a variable
- the word that you put after `var` shouldn't be one of JavaScript's *reserved* words
- you can assign values with a single equal sign `=`
- you can subtract using the `-` sign

## Semantics

Semantics is about the meaning of the sentence constructed by putting together some valid syntax. `+` is valid syntax for plus operator but if you type `x+1` in a javascript program and run it, it will give you an error. Because you are trying to add `1`to a variable that doesn't exist. Yes, `x`is valid syntax, yes, `+`is valid syntax, and yes `1`is valid syntax. But it doesn't mean anything here. Instead something like this could mean something:

```
1   var x =
    1;
2.  x + 1
```

Here, you are saying that `x`is a variable with a value of `1`and you are adding `1`to a it, and it will give you back `2`

## Pragmatics

In programing languages pragmatics are the ways in which the language's features are used to build effective programs. And in JavaScript this is where you will find yourself diving deep into the core concepts of programing and JavaScript. Some examples are:

- what is a variable scope and how can you use it in your programs
- what are closures and how can you use it in your programs
- when do you use arrow functions and when do you use classic functions

What are formal translation models?

Models for the description of the formal translations are syntax-directed translation schemes. The special case of syntax-directed translation schemes are simple syntax-directed translation schemes, which can be written in the form of translation grammars.

# Formal grammar

- Formal grammar is a set of rules. It is used to identify correct or incorrect strings of tokens in a language. The formal grammar is represented as G.
- Formal grammar is used to generate all possible strings over the alphabet that is syntactically correct in the language.
- Formal grammar is used mostly in the syntactic analysis phase (parsing) particularly during the compilation.

Formal grammar G is written as follows:

1. G = <V, N, P, S>

## Where:

**N** describes a finite set of non-terminal symbols.
**V** describes a finite set of terminal symbols.
**P** describes a set of production rules
**S** is the start symbol.

**Example:**

L = {a, b}, N = {S, R, B}

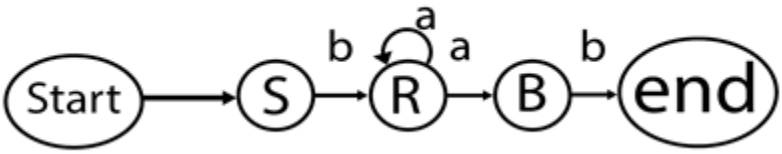
## Production rules:

1. S = bR
2. R = aR
3. R = aB

4. B = b

   Through this production we can produce some strings like: bab, baab, baaab etc.

   This production describes the string of shape $ba^nab$.




**Variables in Programming Languages**

### Variables in Java

```
1. public class JavaDemo {
2. public static void main(String []args) {
3. int a;
4. int b;
5. a = 15;
6. b = 18;
7. System. out. println("Value of a = " + a);
8. System. out. println("Value of b = " + b);
```

## What are the Variables?

Variables are the names that assign to computer memory locations where values are stored in a computer program. A variable is a named data unit with a value assigned with it. Variables are utilized in almost all programming languages and may take many several forms specified by the script or software programmer. The name of the variable denotes the type of data it holds. Variables are so named because the information represented by them can change while the operations on the variable remain constant. In general, a program should be written in a **Symbolic** notation so that a statement is always true symbolically.

Variables can represent any type of data, such as **Booleans, names, sounds, scalars, texts, integers, arrays, images**, or any item or class of objects supported by the computer language. Compilers and interpreters replace the symbolic names of variables with the real data location. During execution, data in locations changes, but locations and names remain constant.

## Creating Variables

In C programming, it is also known as declaring variables. Many programming languages have various methods to declare variables inside the program.

**For example:**

Let's take a simple example of C programming that shows how we may declare a variable in the program

```
1. #include <stdio.h>
2. Int main()
3. {
4. int a;
5. int b;
6. }
```

In the above example, we have declared two variables with the names **a** and **b** to reserve two memory locations. The int keyword was used to describe the variable data type, indicating that we wish to store integer values in these two variables. We may also build variables to store the value of **long, float, char**, or any other data type.

**For example:**

```
1. /* variable to store long value */
2. long a;
3.
4. /* variable to store float value */
5. float b;
6.
7. /* variable to store char value */
8. char c;
```

## Store Values in Variables

As we have seen in the above example, we learn that how we may declare the variable in the program. Now, we are going to discuss how we may store the value in the variables.

**For example:**

```
1. #include <stdio.h>
2. int main()
3. {
4. int a;
```

```
5. int b;
6. a = 15;
7. b = 18
8. }
```

The above program contains two extra statements in which we store **15** in variable **a** and **18** in variable **b**. Almost all programming languages provide a similar method for saving values in variables in which we keep the variable name on the left-hand side of an equal sign = and the value we wish to store in the variable on the right-hand side.

We have now accomplished two steps: we established two variables and then stored the required data in those variables. As a result, variable a now has **a** value of **15**, and variable **b** now has a value of **18**. To put it another way, when the preceding program runs, memory location **a** will hold **15**, and memory location **b** will retain **18**.

# C Expressions

An expression is a formula in which operands are linked to each other by the use of operators to compute a value. An operand can be a function reference, a variable, an array element or a constant.

**Let's see an example:**

```
a-b;
```

In the above expression, minus character (-) is an operator, and a, and b are the two operands.

**There are four types of expressions exist in C:**

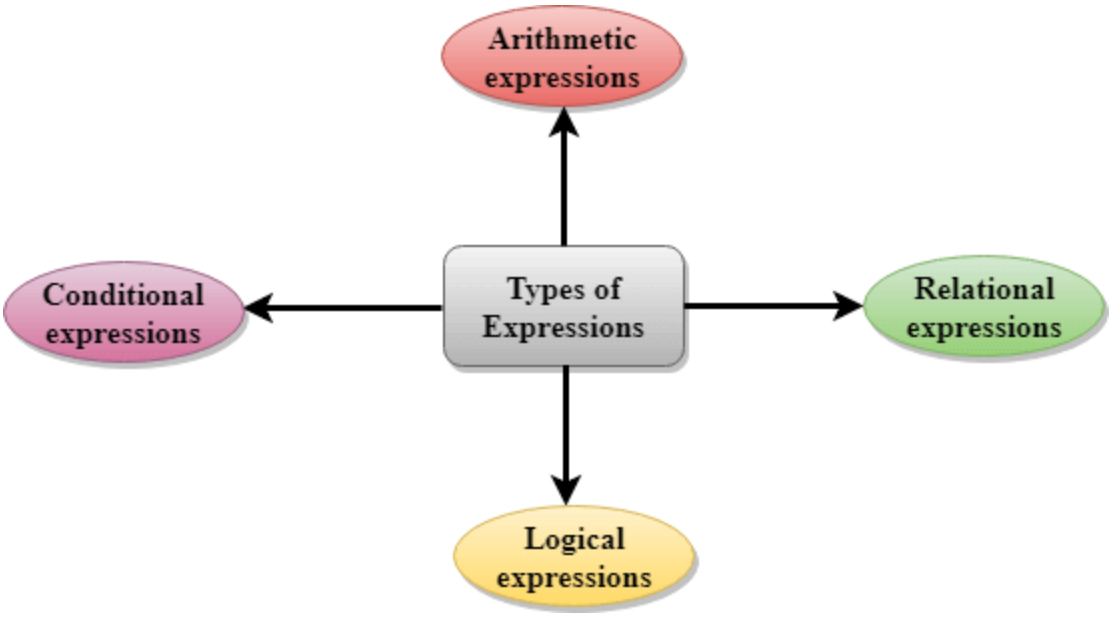
- Arithmetic expressions
- Relational expressions
- Logical expressions
- Conditional expressions

Each type of expression takes certain types of operands and uses a specific set of operators. Evaluation of a particular expression produces a specific value.

**For example:**

```
x = 9/2 + a-b;
```

The entire above line is a statement, not an expression. The portion after the equal is an expression.




# Arithmetic Expressions

An arithmetic expression is an expression that consists of operands and arithmetic operators. An arithmetic expression computes a value of type int, float or double.

When an expression contains only integral operands, then it is known as pure integer expression when it contains only real operands, it is known as pure real expression, and when it contains both integral and real operands, it is known as mixed mode expression.

**Evaluation of Arithmetic Expressions**

The expressions are evaluated by performing one operation at a time. The precedence and associativity of operators decide the order of the evaluation of individual operations.

**When individual operations are performed, the following cases can be happened:**

- When both the operands are of type integer, then arithmetic will be performed, and the result of the operation would be an integer value. For example, 3/2 will yield 1 not 1.5 as the fractional part is ignored.
- When both the operands are of type float, then arithmetic will be performed, and the result of the operation would be a real value. For example, 2.0/2.0 will yield 1.0, not 1.
- If one operand is of type integer and another operand is of type real, then the mixed arithmetic will be performed. In this case, the first operand is converted into a real operand, and then arithmetic is performed to produce the real value. For example, 6/2.0 will yield 3.0 as the first value of 6 is converted into 6.0 and then arithmetic is performed to produce 3.0.

Let's understand through an example.

6*2/ (2+1 * 2/3 + 6) + 8 * (8/4)

| Evaluation of expression | Description of each operation |
|---|---|
| 6*2/( 2+1 * 2/3 +6) +8 * (8/4) | An expression is given. |
| 6*2/(2+2/3 + 6) + 8 * (8/4) | 2 is multiplied by 1, giving value 2. |
| 6*2/(2+0+6) + 8 * (8/4) | 2 is divided by 3, giving value 0. |
| 6*2/ 8+ 8 * (8/4) | 2 is added to 6, giving value 8. |
| 6*2/8 + 8 * 2 | 8 is divided by 4, giving value 2. |
| 12/8 +8 * 2 | 6 is multiplied by 2, giving value 12. |
| 1 + 8 * 2 | 12 is divided by 8, giving value 1. |
| 1 + 16 | 8 is multiplied by 2, giving value 16. |
| 17 | 1 is added to 16, giving value 17. |

## Relational Expressions

- A relational expression is an expression used to compare two operands.
- It is a condition which is used to decide whether the action should be taken or not.
- In relational expressions, a numeric value cannot be compared with the string value.
- The result of the relational expression can be either zero or non-zero value. Here, the zero value is equivalent to a false and non-zero value is equivalent to true.

| Relational Expression | Description |
|---|---|
| x%2 = = 0 | This condition is used to check whether the x is an even number or not. The relation expression results in value 1 if x is an even number otherwise results in value 0. |
| a!=b | It is used to check whether a is not equal to b. This relational expression results in 1 if a not equal to b otherwise 0. |
| a+b = = x+y | It is used to check whether the expression "a+b" is equal to the expression "x+y". |

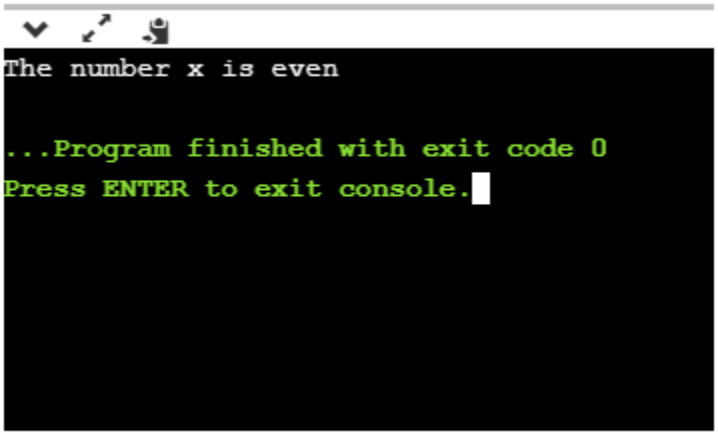
| | |
|---|---|
| a>=9 | It is used to check whether the value of a is greater than or equal to 9. |

**Let's see a simple example:**

```
#include <stdio.h>
 int main()
{

    int x=4;
    if(x%2==0)
    {
        printf("The number x is even");
    }
    else
    printf("The number x is not even");
    return 0;
}
```

**Output**




# Logical Expressions

- A logical expression is an expression that computes either a zero or non-zero value.
- It is a complex test condition to take a decision.

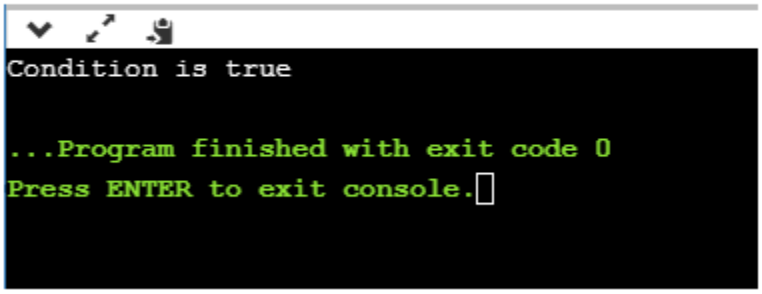
**Let's see some example of the logical expressions.**

| Logical Expressions | Description |
|---|---|
| ( x > 4 ) && ( x < 6 ) | It is a test condition to check whether the x is greater than 4 and x is less than 6. The resu of the condition is true only when both the conditions are true. |
| x > 10 \|\| y <11 | It is a test condition used to check whether x is greater than 10 or y is less than 11. Th result of the test condition is true if either of the conditions holds true value. |
| ! ( x > 10 ) && ( y = = 2 ) | It is a test condition used to check whether x is not greater than 10 and y is equal to 2. Th result of the condition is true if both the conditions are true. |

**Let's see a simple program of "&&" operator.**

```
1. #include <stdio.h>
2. int main()
3. {
4.    int x = 4;
5.    int y = 10;
6.    if ( (x <10) && (y>5))
7.    {
8.       printf("Condition is true");
9.    }
10.   else
11.   printf("Condition is false");
12.   return 0;
13. }
```

**Output**




## Conditional Expressions

- A conditional expression is an expression that returns 1 if the condition is true otherwise 0.
- A conditional operator is also known as a ternary operator.

**The Syntax of Conditional operator**

**Suppose exp1, exp2 and exp3 are three expressions.**

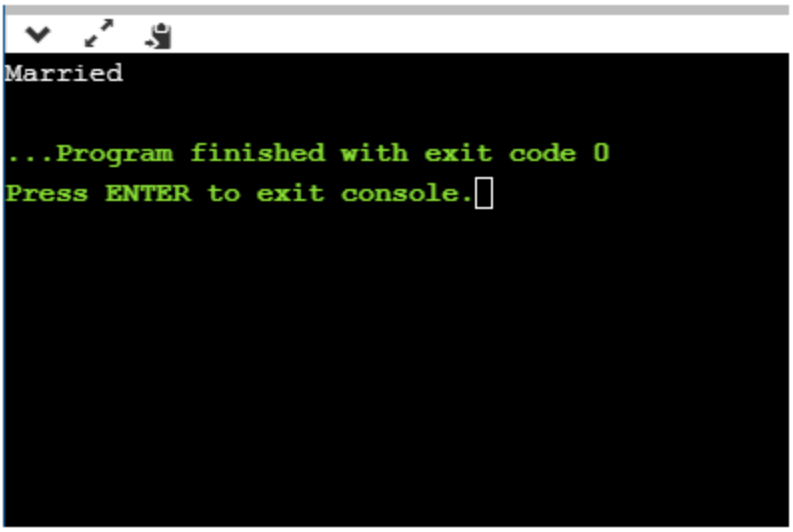
exp1 ? exp2 : exp3

The above expression is a conditional expression which is evaluated on the basis of the value of the exp1 expression. If the condition of the expression exp1 holds true, then the final conditional expression is represented by exp2 otherwise represented by exp3.

**Let's understand through a simple example.**

```
1. #include<stdio.h>
2. #include<string.h>
3. int main()
4. {
5.     int age = 25;
6.     char status;
7.     status = (age>22) ? 'M': 'U';
8.     if(status == 'M')
9.     printf("Married");
10.    else
11.    printf("Unmarried");
12.    return 0;
13. }
```

**Output**




# Types of Statements in Java

**Statements** are roughly equivalent to sentences in natural languages. In general, statements are just like English sentences that make valid sense. In this section, we will discuss **what is a statement in Java** and the **types of statements in Java**.

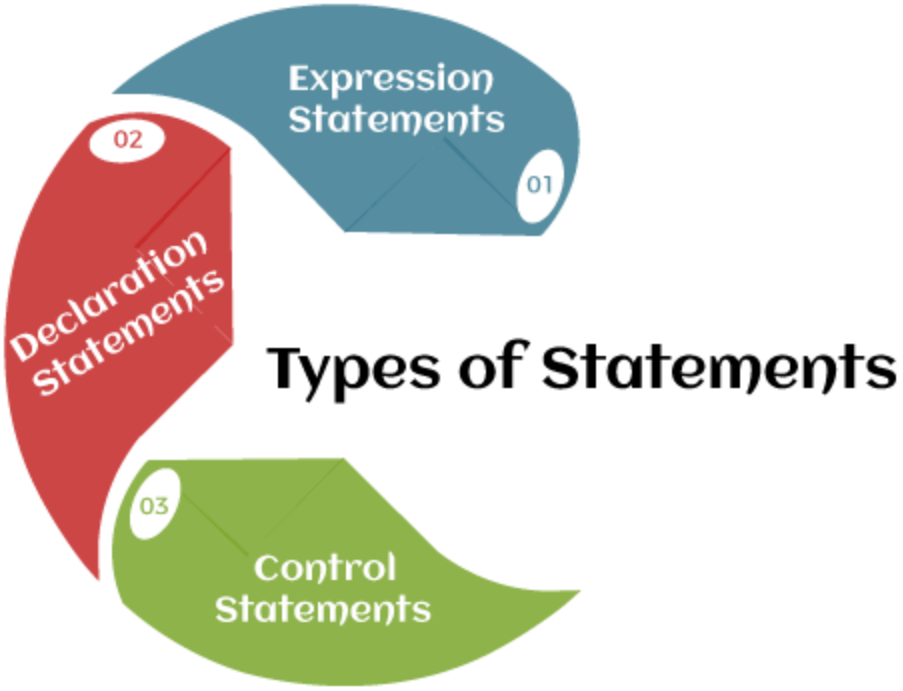
## What is statement in Java?

In Java, a **statement** is an executable instruction that tells the compiler what to perform. It forms a complete command to be executed and can include one or more expressions. A sentence forms a complete idea that can include one or more clauses.

## Types of Statements

Java statements can be broadly classified into the following categories:

- Expression Statements
- Declaration Statements
- Control Statements




## Expression Statements

Expression is an essential building block of any Java program. Generally, it is used to generate a new value. Sometimes, we can also assign a value to a variable. In Java, expression is the combination of values, variables, operators, and method calls.

There are three types of expressions in Java:

- Expressions that **produce** a value. For example, **(6+9), (9%2), (pi*radius) + 2**. Note that the expression enclosed in the parentheses will be evaluate first, after that rest of the expression.
- Expressions that **assign** a value. For example, **number = 90, pi = 3.14.**
- Expression that **neither produces any result nor assigns a value**. For example, **increment** or **decrement** a value by using increment or decrement operator respectively, **method invocation**, etc. These expressions modify the value of a variable or state (memory) of a program. For example, **count++, int sum = a + b**; The expression changes only the value of the variable **sum**. The value of variables **a** and **b** do not change, so it is also a side effect.

## Declaration Statements

In declaration statements, we declare variables and constants by specifying their data type and name. A variable holds a value that is going to use in the Java program. For example:

```
1. int quantity;
2. boolean flag;
3. String message;
```

Also, we can initialize a value to a variable. For example:

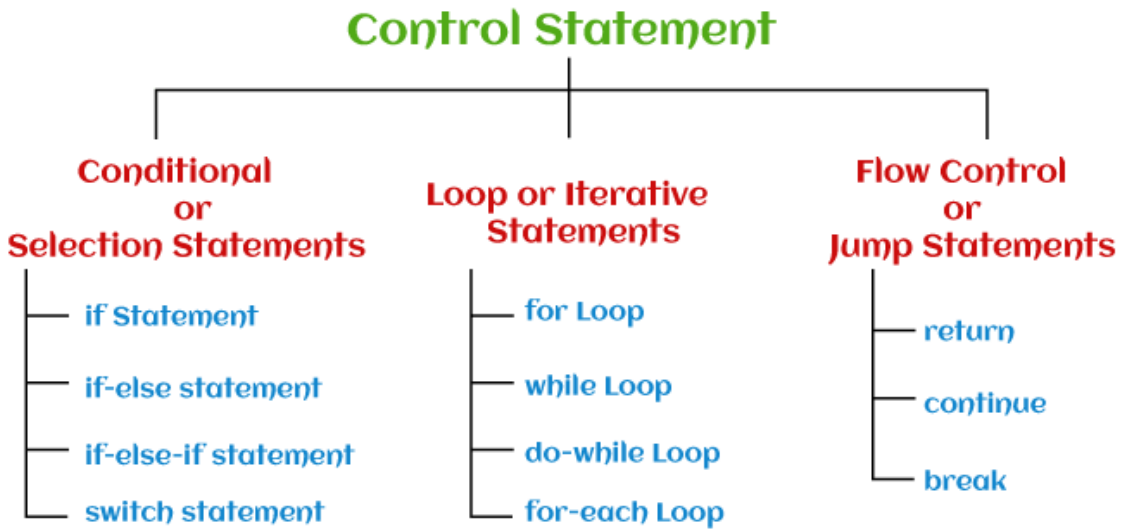
```
1. int quantity = 20;
2. boolean flag = false;
3. String message = "Hello";
```

Java also allows us to declare multiple variables in a single declaration statement. Note that all the variables must be of the same data type.

```
1. int quantity, batch_number, lot_number;
2. boolean flag = false, isContains = true;
3. String message = "Hello", how are you;
```

## Control Statement

Control statements decide the flow (order or sequence of execution of statements) of a Java program. In Java, statements are parsed from top to bottom. Therefore, using the control flow statements can interrupt a particular section of a program based on a certain condition.




There are the following types of control statements:

1. **Conditional or Selection Statements**
    - if Statement
    - if-else statement
    - if-else-if statement
    - switch statement
2. **Loop or Iterative Statements**
    - for Loop
    - while Loop
    - do-while Loop
    - for-each Loop
3. **Flow Control or Jump Statements**
    - return
    - continue
    - break

## Example of Statement

```
1. //declaration statement
2. int number;
3. //expression statement
4. number = 412;
5. //control flow statement
6. if (number > 10 )
7. {
8. //expression statement
9. System.out.println(number + " is greater than 100");
10. }
```

# binding in C++

The binding which can be resolved by the compiler using runtime is known as static binding. For example, all the final, static, and private methods are bound at run time. All the overloaded methods are binded using static binding.

The concept of dynamic binding removed the problems of static binding.

**Dynamic binding**

The general meaning of binding is linking something to a thing. Here linking of objects is done. In a programming sense, we can describe binding as linking function definition with the function call.

So the term dynamic binding means to select a particular function to run until the runtime. Based on the type of object, the respective function will be called.

As dynamic binding provides flexibility, it avoids the problem of static binding as it happened at compile time and thus linked the function call with the function definition.

**Use of dynamic binding**

On that note, dynamic binding also helps us to handle different objects using a single function name. It also reduces the complexity and helps the developer to debug the code and errors.

**How to implement dynamic binding?**

The concept of dynamic programming is implemented with virtual functions.

**Virtual functions**

A function declared in the base class and overridden(redefined) in the child class is called a virtual function. When we refer derived class object using a pointer or reference to the base, we can call a virtual function for that object and execute the derived class's version of the function.

**Characteristics**

- Run time function resolving
- Used to achieve runtime polymorphism
- All virtual functions are declared in the base class
- Assurance of calling correct function for an object regardless of the pointer(reference) used for function call.
- A virtual function cannot be declared as static
- No virtual constructor exists but a virtual destructor can be made.
- Virtual function definition should be the same in the base as well as the derived class.
- A virtual function can be a friend of another class.
- The definition is always in the base class and overrides in the derived class

Now let us see the following problem that occurs without virtual keywords.

**Example**

Let us take a class A with a function **final_print()**, and class B inherits A publicly. B also has its **final_print()** function.

If we make an object of A and call **final_print()**, it will run of base class whereas, if we make an object of B and call final_print(), it will run of base only.

**Code**

```
#include <iostream>
using namespace std;
class A {
public:
    void final_print() // function that call display
    {
display();
    }
    void display() // the display function
    {
cout<< "Printing from the base class" <<endl;
    }
};
class B : public A // B inherit a publicly
{
public:
    void display() // B's display
    {
cout<< "Printing from the derived class" <<endl;
    }
};
int main()
{
    A obj1; // Creating A's pbject
    obj1.final_print(); // Calling final_print
    B obj2; // calling b
    obj2.final_print();
    return 0;
}
```

**Output**

```
Printing from the base class
Printing from the base class
```

# What are the Variables?

Variables are the names that assign to computer memory locations where values are stored in a computer program. A variable is a named data unit with a value assigned with it. Variables are utilized in almost all programming languages and may take many several forms specified by the script or software programmer. The name of the variable denotes the type of data it holds. Variables are so named because the information represented by them can change while the operations on the

variable remain constant. In general, a program should be written in a **Symbolic** notation so that a statement is always true symbolically.

Variables can represent any type of data, such as **Booleans, names, sounds, scalars, texts, integers, arrays, images**, or any item or class of objects supported by the computer language. Compilers and interpreters replace the symbolic names of variables with the real data location. During execution, data in locations changes, but locations and names remain constant.

## Creating Variables

In C programming, it is also known as declaring variables. Many programming languages have various methods to declare variables inside the program.

**For example:**

Let's take a simple example of C programming that shows how we may declare a variable in the program

```
1. #include <stdio.h>
2. Int main()
3. {
4. int a;
5. int b;
6. }
```

In the above example, we have declared two variables with the names **a** and **b** to reserve two memory locations. The int keyword was used to describe the variable data type, indicating that we wish to store integer values in these two variables. We may also build variables to store the value of **long, float, char**, or any other data type.

**For example:**

```
1. /* variable to store long value */
2. long a;
3.
4. /* variable to store float value */
5. float b;
6.
7. /* variable to store char value */
8. char c;
```

## Store Values in Variables

As we have seen in the above example, we learn that how we may declare the variable in the program. Now, we are going to discuss how we may store the value in the variables.

**For example:**

```
#include <stdio.h>
int main()
{
int a;
int b;
a = 15;
b = 18
}
```

The above program contains two extra statements in which we store **15** in variable **a** and **18** in variable **b**. Almost all programming languages provide a similar method for saving values in variables in which we keep the variable name on the left-hand side of an equal sign = and the value we wish to store in the variable on the right-hand side.

We have now accomplished two steps: we established two variables and then stored the required data in those variables. As a result, variable a now has **a** value of **15**, and variable **b** now has a value of **18**. To put it another way, when the preceding program runs, memory location **a** will hold **15**, and memory location **b** will retain **18**.

# C++ Expression

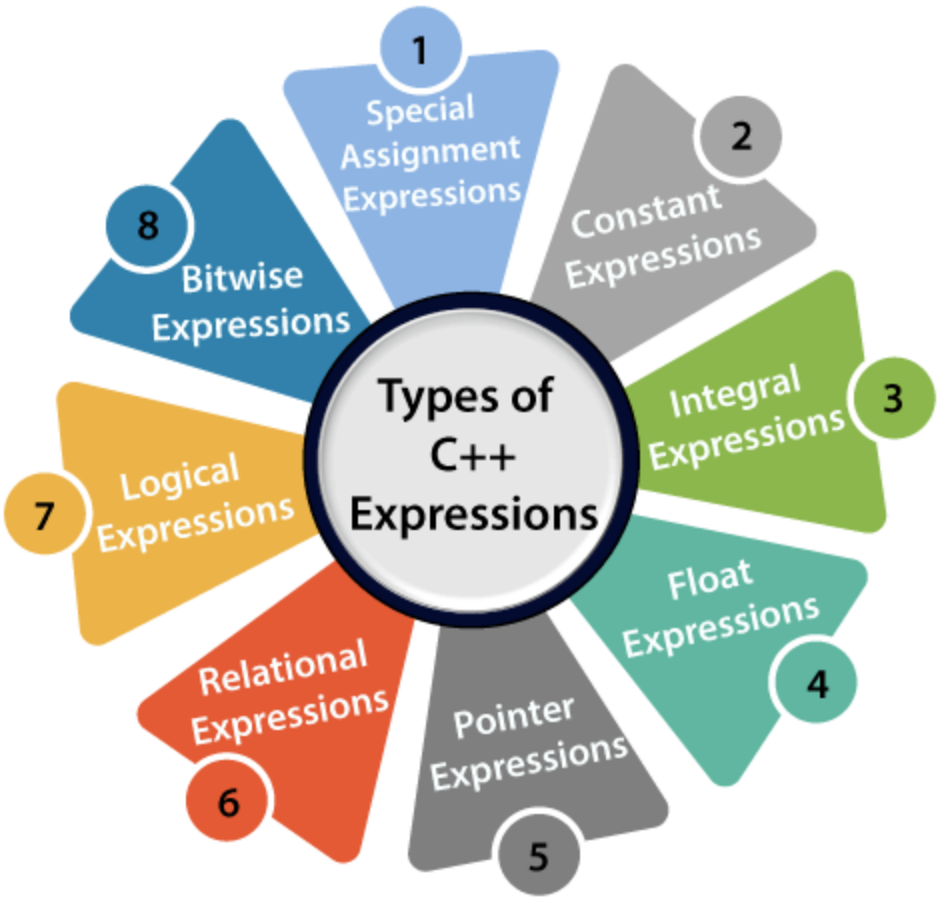
C++ expression consists of operators, constants, and variables which are arranged according to the rules of the language. It can also contain function calls which return values. An expression can consist of one or more operands, zero or more operators to compute a value. Every expression produces some value which is assigned to the variable with the help of an assignment operator.

**Examples of C++ expression:**

1. (a+b) - c
2. (x/y) -z
3. 4a2 - 5b +c
4. (a+b) * (x+y)

## An expression can be of following types:

- Constant expressions
- Integral expressions
- Float expressions
- Pointer expressions
- Relational expressions
- Logical expressions
- Bitwise expressions
- Special assignment expressions




## Constant expressions

A constant expression is an expression that consists of only constant values. It is an expression whose value is determined at the compile-time but evaluated at the run-time. It can be composed of integer, character, floating-point, and enumeration constants.

Constants are used in the following situations:

- It is used in the subscript declarator to describe the array bound.
- It is used after the case keyword in the switch statement.
- It is used as a numeric value in an **enum**
- It specifies a bit-field width.
- It is used in the pre-processor **#if**

In the above scenarios, the constant expression can have integer, character, and enumeration constants. We can use the static and extern keyword with the constants to define the function-scope.

The following table shows the expression containing constant value:

| Expression containing constant | Constant value |
|---|---|
| x = (2/3) * 4 | (2/3) * 4 |
| extern int y = 67 | 67 |
| int z = 43 | 43 |
| static int a = 56 | 56 |

## Integral Expressions

An integer expression is an expression that produces the integer value as output after performing all the explicit and implicit conversions.

**Following are the examples of integral expression:**

1. (x * y) -5
2. x + **int**(9.0)
3. where x and y are the integers.

## Float Expressions

A float expression is an expression that produces floating-point value as output after performing all the explicit and implicit conversions.

The following are the examples of float expressions:

1. x+y
2. (x/10) + y
3. 34.5
4. x+**float**(10)

## Pointer Expressions

A pointer expression is an expression that produces address value as an output.

**The following are the examples of pointer expression:**

1. &x

2. ptr
3. ptr++
4. ptr-

## Relational Expressions

A relational expression is an expression that produces a value of type bool, which can be either true or false. It is also known as a boolean expression. When arithmetic expressions are used on both sides of the relational operator, arithmetic expressions are evaluated first, and then their results are compared.

**The following are the examples of the relational expression:**

1. a>b
2. a-b >= x-y
3. a+b>80

## Logical Expressions

A logical expression is an expression that combines two or more relational expressions and produces a bool type value. The logical operators are '&&' and '||' that combines two or more relational expressions.

The following are some examples of logical expressions:

1. a>b && x>y
2. a>10 || b==5

## Bitwise Expressions

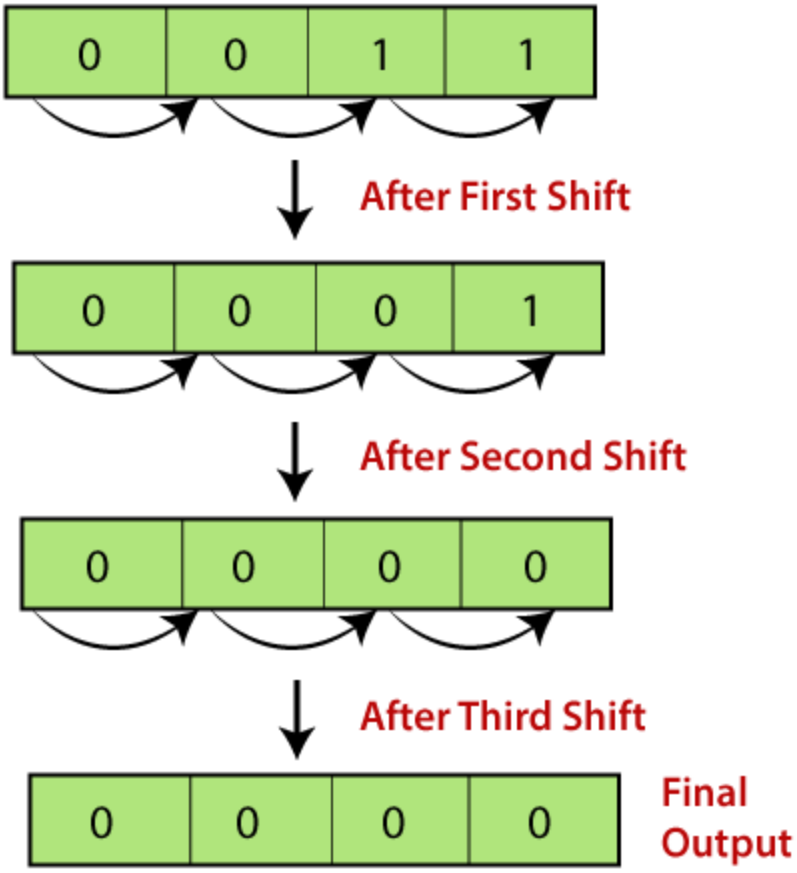
A bitwise expression is an expression which is used to manipulate the data at a bit level. They are basically used to shift the bits.

For example:

x=3

x>>3 // This statement means that we are shifting the three-bit position to the right.

In the above example, the value of 'x' is 3 and its binary value is 0011. We are shifting the value of 'x' by three-bit position to the right. Let's understand through the diagrammatic representation.




## Special Assignment Expressions

Special assignment expressions are the expressions which can be further classified depending upon the value assigned to the variable.

- **Chained Assignment**

Chained assignment expression is an expression in which the same value is assigned to more than one variable by using single statement.

**For example:**

1. a=b=20
2. or

3. (a=b) = 20

- **Embedded Assignment Expression**

An embedded assignment expression is an assignment expression in which assignment expression is enclosed within another assignment expression.

- **Compound Assignment**

A compound assignment expression is an expression which is a combination of an assignment operator and binary operator.

**For example,**

1. a+=10;

# Assignment Operator in C

There are different kinds of the operators, such as arithmetic, relational, bitwise, assignment, etc., in the C programming language. The assignment operator is used to assign the value, variable and function to another variable. Let's discuss the various types of the assignment operators such as =, +=, -=, /=, *= and %=.

**Example of the Assignment Operators:**

```
A = 5; // use Assignment symbol to assign 5 to the operand A
B = A; // Assign operand A to the B
B = &A; // Assign the address of operand A to the variable B
A = 20 \ 10 * 2 + 5; // assign equation to the variable A
```

## Simple Assignment Operator (=):

It is the operator used to assign the right side operand or variable to the left side variable.

**Syntax**

```
int a = 5;
or int b = a;
ch = 'a';
```

## Plus and Assign Operator (+=):

The operator is used to add the left side operand to the left operand and then assign results to the left operand.

**Syntax**

1. A += B;
2. Or
3. A = A + B;

## Subtract and Assign Operator (-=):

The operator is used to subtract the left operand with the right operand and then assigns the result to the left operand.

**Syntax**

1. A -= B;
2. Or
3. A = A - B;

## Multiply and Assign Operator (*=)

The operator is used to multiply the left operand with the right operand and then assign result to the left operand.

**Syntax**

1. A *= B;
2. Or
3. A = A * B;

## Divide and Assign Operator (/=):

An operator is used between the left and right operands, which divides the first number by the second number to return the result in the left operand.

**Syntax**

1. A /= B;
2. Or
3. A = A / B;

## Modulus and Assign Operator (%=):

An operator used between the left operand and the right operand divides the first number (n1) by the second number (n2) and returns the remainder in the left operand.

**Syntax**

1. A %= B;
2. Or
3. A = A % B;

# Lvalue and Rvalue in C

## What is lvalue?

***Lvalue*** merely denotes a memory ***location-identifiable item*** (i.e., one with an address). Any ***assignment statement*** must allow for the storage of data in ***"lvalue"***. A function, an expression (such as ***a+b***), or a constant (such as ***3, 4***, etc.) cannot be a ***lvalue***.

The term ***"l-value"*** describes a ***memory location*** that is used to ***identify*** an object. The ***left*** or ***right side*** of the ***assignment operator (=)*** may contain a ***l-value. L-value*** is frequently used as ***identification. "Modifiable l-values"*** are expressions that refer to customizable places. An ***array type***, an incomplete type, or a type with the const attribute is all prohibited for a changeable l-value. There cannot be any members with the ***const attribute*** in structures or unions in order for them to be editable ***lvalues***.

The value of the variable is the ***value*** that is stored at the location designated by the name of the ***identifier***. If an ***identifier*** relates to a ***memory location*** and its type is ***arithmetic, structure, union***, or ***pointer***, it qualifies as a changeable ***lvalue***. For instance, ****ptr*** is a changeable ***l-value*** that identifies the storage region to which ptr points if ***ptr*** is a ***pointer*** to a storage region. Expressions that locate (designate) objects were given the new moniker of ***"locator value"*** in C. One of the following describes the l-value:

- Any variable's name, such as an ***identifier*** for an ***integral***, ***floating-point, pointer, structure,*** or ***union type***.
- An expression for the ***subscript ([])*** that does not evaluate an array.
- An expression with a ***unary indirection (*)*** that doesn't make reference to an array
- An enclosed ***l-value***
- A const object (an ***l-value*** that cannot be changed).
- Indirection through a pointer that, if it's not a function pointer, produces the desired effect.
- ***Member access*** by ***pointer(-> or.)*** results in.

## Example:

```
1. #include <stdio.h>
2.
```

```c
int main() {
    int a; // declare 'a' as an object of type 'int'
    int b;

    // 'a' is an expression referring to an 'int' object as an l-value
    a = 1;

    b = a; // assigning the value of 'a' to 'b'

printf("The value of 'a' is %d\n", a);
printf("The value of 'b' is %d\n", b);

    // The following line will cause a compilation error
    // 9 = a;

    return 0;
}
```

**Output:**

```
The value of 'a' is 1
The value of 'b' is 1
```

**Explanation:**

In this example, ***a*** and ***b*** are declared as ***integer variables***. A value of ***1*** is given to ***a***, and then ***b*** is given that ***same value***. The result will indicate that both ***a*** and ***b*** have values of *1*.

Because a ***l-value*** is anticipated on the ***left side*** of the ***assignment operator (=)***, the line ***9 = a***; will result in a ***compilation error***. In this example, it is not an acceptable ***l-value*** since the ***literal 9*** cannot be given a new value.

# What is r-value?

***R-value*** simply refers to an object without an address or a location that can be found in memory. It can produce a ***constant expression*** or ***value*** as a return value. A simple expression like ***a+b*** will yield a constant.

The term ***"r-value"*** describes a data item that is kept in memory at a ***specific address***. A ***r-value*** is an expression that cannot have a value ***assigned*** to it. Hence it can only occur on the ***right side*** of the ***assignment operator (=)***, not the left.

## Example:

```c
#include <stdio.h>

int main() {
    int a = 1, b;
    int *p, *q;

    // The following line will cause a compilation error
    // a + 1 = b;

    p = &a; // assigning the address of 'a' to 'p'
    *p = 1; // assigning a value to the memory location pointed by 'p'

    // The following line will cause a compilation error
    // p + 2 = 18;

    q = p + 5; // assigning the address calculated by 'p + 5' to 'q'

    *(p + 2) = 18; // assigning a value to the memory location calculated by 'p + 2'

    p = &b; // assigning the address of 'b' to 'p'

    int arr[20];
arr[12] = 42; // assigning a value to the element at index 12 of 'arr'

    struct S { int m; };
    struct S obj;
    struct S* ptr = &obj;

ptr->m = 24; // assigning a value to the member 'm' of the struct pointed by 'ptr'

printf("The value of 'a' is %d\n", a);
printf("The value of 'b' is %d\n", b);
printf("The value at index 12 of 'arr' is %d\n", arr[12]);
printf("The value of 'obj.m' is %d\n", obj.m);

    return 0;
}
```

**Output:**

```
The value of 'a' is 1
The value of 'b' is 0
The value at index 12 of 'arr' is 42
The value of 'obj.m' is 24
```

**Environment in Programming Languages**

Environment Setup is not an element of any Programming Language, it is the first step to be followed before setting on to write a program. A Web browser such as Internet Explorer, Chrome, Safari, etc.

Though Environment Setup is not an element of any Programming Language, it is the first step to be followed before setting on to write a program.

When we say Environment Setup, it simply implies a base on top of which we can do our programming. Thus, we need to have the required software setup, i.e., installation on our PC which will be used to write computer programs, compile, and execute them. For example, if you need to browse Internet, then you need the following setup on your machine −

- A working Internet connection to connect to the Internet
- A Web browser such as Internet Explorer, Chrome, Safari, etc.

If you are a PC user, then you will recognize the following screenshot, which we have taken from the Internet Explorer while browsing tutorialspoint.com.


Similarly, you will need the following setup to start with programming using any programming language.

- A text editor to create computer programs.
- A compiler to compile the programs into binary format.
- An interpreter to execute the programs directly.

In case you don’t have sufficient exposure to computers, you will not be able to set up either of these software. So, we suggest you take the help from any technical person around you to set up the programming environment on your machine from where you can start. But for you, it is important to understand what these items are.

#### Stores in Programming Language

The computer system is simply a machine and hence it cannot perform any work; therefore, in order to make it functional different

languages are developed, which are known as programming languages or simply computer languages.

Over the last two decades, dozens of computer languages have been developed. Each of these languages comes with its own set of vocabulary and rules, better known as syntax. Furthermore, while writing the computer language, syntax has to be followed literally, as even a small mistake will result in an error and not generate the required output.

Following are the major categories of Programming Languages –

- Machine Language
- Assembly Language
- High Level Language
- System Language
- Scripting Language

Let us discuss the programming languages in brief.

## Machine Language or Code

This is the language that is written for the computer hardware. Such language is effected directly by the central processing unit (CPU) of a computer system.

## Assembly Language

It is a language of an encoding of machine code that makes simpler and readable.

## High Level Language

The high level language is simple and easy to understand and it is similar to English language. For example, COBOL, FORTRAN, BASIC, C, C+, Python, etc.

High-level languages are very important, as they help in developing complex software and they have the following advantages –

- Unlike assembly language or machine language, users do not need to learn the high-level language in order to work with it.
- High-level languages are similar to natural languages, therefore, easy to learn and understand.
- High-level language is designed in such a way that it detects the errors immediately.
- High-level language is easy to maintain and it can be easily modified.
- High-level language makes development faster.
- High-level language is comparatively cheaper to develop.
- High-level language is easier to document.

Although a high-level language has many benefits, yet it also has a drawback. It has poor control on machine/hardware.

The following table lists down the frequently used languages –

| |
|---|
| SQL |
| Java |
| Javascript |
| C# |
| Python |
| C++ |
| PHP |
| IOS |
| Ruby/Rails |
| .Net |

# Storage Allocation

The different ways to allocate memory are:

1. Static storage allocation

2. Stack storage allocation
3. Heap storage allocation

## Static storage allocation

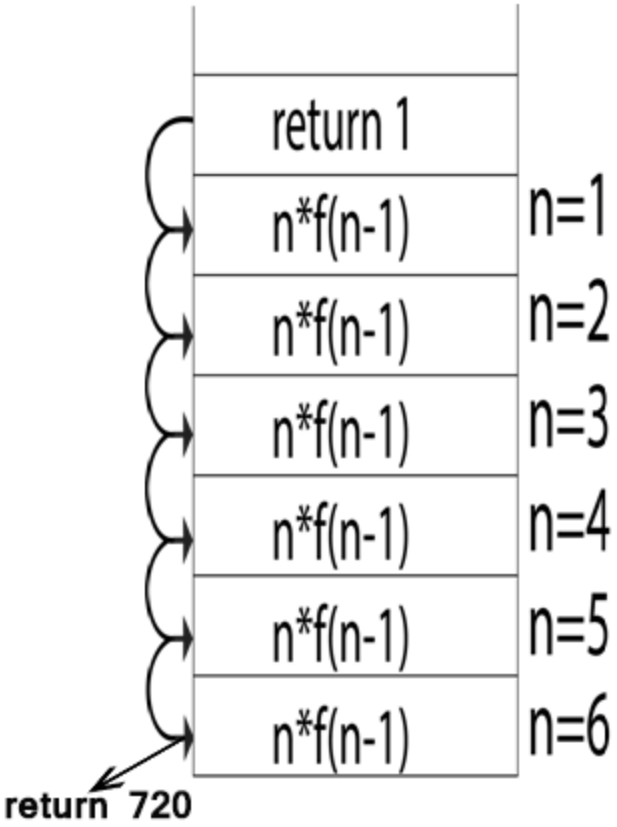
- In static allocation, names are bound to storage locations.
- If memory is created at compile time then the memory will be created in static area and only once.
- Static allocation supports the dynamic data structure that means memory is created only at compile time and deallocated after program completion.
- The drawback with static storage allocation is that the size and position of data objects should be known at compile time.
- Another drawback is restriction of the recursion procedure.

## Stack Storage Allocation

- In static storage allocation, storage is organized as a stack.
- An activation record is pushed into the stack when activation begins and it is popped when the activation end.
- Activation record contains the locals so that they are bound to fresh storage in each activation record. The value of locals is deleted when the activation ends.
- It works on the basis of last-in-first-out (LIFO) and this allocation supports the recursion process.

## Heap Storage Allocation

- Heap allocation is the most flexible allocation scheme.
- Allocation and deallocation of memory can be done at any time and at any place depending upon the user's requirement.
- Heap allocation is used to allocate memory to the variables dynamically and when the variables are no more used then claim it back.
- Heap storage allocation supports the recursion process.




## Constant variable definition

Constant is a value that cannot be reassigned. A constant is immutable and cannot be changed. There are some methods in different programming languages to define a constant variable. Most uses the **const** keyword. Using a **const** keyword indicates that the data is read-only. We can use the **const** keyword in programming languages such as C, C++, JavaScript, PHP, etc. It can be applied to declare an object. Java does not directly support the constants. The alternative way to define the constants in java is by using the non-static modifiers **static** and **final**.

The keyword **const** creates a read-only reference to the value. It can neither be re-declared nor can't a value be reassigned to it. That means a const variable or its value can't be changed in a program.

# Variable initialization in C++

Variables are the names given by the user. A datatype is also used to declare and initialize a variable which allocates memory to that variable. There are several datatypes like int, char, float etc. to allocate the memory to that variable.

There are two ways to initialize the variable. One is static initialization in which the variable is assigned a value in the program and another is dynamic initialization in which the variables is assigned a value at the run time.

The following is the syntax of variable initialization.

datatype variable_name = value;

Here,

**datatype** – The datatype of variable like int, char, float etc.

**variable_name** – This is the name of variable given by user.

**value** – Any value to initialize the variable. By default, it is zero.

The following is an example of variable initialization.

## Example

Live Demo

```
#include <iostream>
using namespace std;
int main() {
   int a = 20;
   int b;
   cout << "The value of variable a : "<< a; // static
initialization
   cout << "\nEnter the value of variable b : "; //
dynamic initialization
   cin >> b;
   cout << "\nThe value of variable b : "<< b;
   return 0;
}
```

## Output

The value of variable a : 20

Enter the value of variable b : 28
The value of variable b : 28

# Java Control Statements | Control Flow in Java

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear. However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.

Java provides three types of control flow statements.

1. Decision Making statements
    - if statements
    - switch statement
2. Loop statements
    - do while loop
    - while loop
    - for loop
    - for-each loop
3. Jump statements
    - break statement
    - continue statement

## Decision-Making statements:

As the name suggests, decision-making statements decide which statement to execute and when. Decision-making statements evaluate the Boolean expression and control the program flow depending upon the result of the condition provided. There are two types of decision-making statements in Java, i.e., If statement and switch statement.

## 1) If Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. Simple if statement
2. if-else statement
3. if-else-if ladder
4. Nested if-statement

Let's understand the if-statements one by one.

## 1) Simple if statement:

It is the most basic statement among all control flow statements in Java. It evaluates a Boolean expression and enables the program to enter a block of code if the expression evaluates to true.

Syntax of if statement is given below.

```
1. if(condition) {
2. statement 1; //executes when condition is true
3. }
```

## if-else statement

The if-else statement is an extension to the if-statement, which uses another block of code, i.e., else block. The else block is executed if the condition of the if-block is evaluated as false.

**Syntax:**

```
1. if(condition) {
2. statement 1; //executes when condition is true
3. }
4. else{
5. statement 2; //executes when condition is false
6. }
```

Consider the following example.

**Student.java**

```
1. public class Student {
2. public static void main(String[] args) {
3. int x = 10;
4. int y = 12;
5. if(x+y < 10) {
6. System.out.println("x + y is less than      10");
7. }   else {
8. System.out.println("x + y is greater than 20");
9. }
10. }
11. }
```

**Output:**

```
x + y is greater than 20
```

### ) if-else-if ladder:

The if-else-if statement contains the if-statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else statements that create a decision tree where the program may enter in the block of code where the condition is true. We can also define an else statement at the end of the chain.

```
1. if(condition 1) {
2. statement 1; //executes when condition 1 is true
3. }
4. else if(condition 2) {
5. statement 2; //executes when condition 2 is true
6. }
7. else {
8. statement 2; //executes when all the conditions are false
9. }
```

### Nested if-statement

In nested if-statements, the if statement can contain a **if** or **if-else** statement inside another if or else-if statement.

Syntax of Nested if-statement is given below.

```
1. if(condition 1) {
2. statement 1; //executes when condition 1 is true
3. if(condition 2) {
4. statement 2; //executes when condition 2 is true
5. }
6. else{
7. statement 2; //executes when condition 2 is false
8. }
9. }
```

## Switch Statement:

In Java, Switch statements are similar to if-else-if statements. The switch statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched. The switch statement is easier to use instead of if-else-if statements. It also enhances the readability of the program.

In Java, we have three types of loops that execute similarly. However, there are differences in their syntax and condition checking time.

1. for loop
2. while loop
3. do-while loop

Let's understand the loop statements one by one.

## Java for loop

In Java, for loop is similar to C and C++. It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. We use the for loop only when we exactly know the number of times, we want to execute the block of code.

## Java for-each loop

Java provides an enhanced for loop to traverse the data structures like array or collection. In the for-each loop, we don't need to update the loop variable. The syntax to use the for-each loop in java is given below.

```
for(data_type var : array_name/collection_name){
//statements
}
```

Consider the following example to understand the functioning of the for-each loop in Java.

**Calculation.java**

```
public class Calculation {
public static void main(String[] args) {
// TODO Auto-generated method stub
String[] names = {"Java","C","C++","Python","JavaScript"};
System.out.println("Printing the content of the array names:\n");
for(String name:names) {
System.out.println(name);
}
}
}
```

**Output:**

```
Printing the content of the array names:

Java
C
C++
```

```
Python
JavaScript
```

## Java while loop

The while loop is also used to iterate over the number of statements multiple times. However, if we don't know the number of iterations in advance, it is recommended to use a while loop. Unlike for loop, the initialization and increment/decrement doesn't take place inside the loop statement in while loop.

It is also known as the entry-controlled loop since the condition is checked at the start of the loop. If the condition is true, then the loop body will be executed; otherwise, the statements after the loop will be executed.

The syntax of the while loop is given below.

```
1. while(condition){
2. //looping statements
3. }
```

## Java do-while loop

The do-while loop checks the condition at the end of the loop after executing the loop statements. When the number of iteration is not known and we have to execute the loop at least once, we can use do-while loop.

It is also known as the exit-controlled loop since the condition is not checked in advance.

## Jump Statements

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program. There are two types of jump statements in Java, i.e., break and continue.

## Java break statement

As the name suggests, the break statement is used to break the current flow of the program and transfer the control to the next statement outside a loop or switch statement. However, it breaks only the inner loop in the case of the nested loop.

The break statement cannot be used independently in the Java program, i.e., it can only be written inside the loop or switch statement.

**The break statement example with for loop**

Consider the following example in which we have used the break statement with the for loop.

## Java continue statement

Unlike break statement, the continue statement doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

Consider the following example to understand the functioning of the continue statement in Java.

```
1.  public class ContinueExample {
2.
3.  public static void main(String[] args) {
4.  // TODO Auto-generated method stub
5.
6.  for(int i = 0; i<= 2; i++) {
7.
8.  for (int j = i; j<=5; j++) {
9.
10. if(j == 4) {
11. continue;
12. }
13. System.out.println(j);
14. }
15. }
16. }
17.
18. }
```

**Output:**

```
0
1
2
3
5
1
2
3
5
2
3
5
```